Projet MLSF – Un solveur SAT pour Meowdoku

Partie I

Pablo Bosch Cécile Chevrière

Octobre 2026

Organisation du rendu. Les programmes se trouvent dans le dossier `Session de travail` : le module `meowdoku.py` (lecture des grilles, encodage CNF, écriture DIMACS, vérification), le module `generateur.py` (génération de grilles), le module `dpll.py` (lecture DIMACS, algorithmes DPLL 1 et 2), le programme `resoudre.py` (résolution d'une grille) et le notebook `projet_partie1.ipynb`, qui exécute les programmes dans l'ordre des questions. Les grilles et les fichiers DIMACS produits sont rangés dans `grilles/` et `cnf/`, les fichiers solutions dans `solutions/`.

1 Question 1 – Modélisation de Meowdoku

1.1 Variables

Pour une grille $N \times M$ découpée en régions $R_1, \dots, R_K$, on introduit une variable $x_{i,j}$ par case $(i,j) \in \{1,\dots,N\} \times \{1,\dots,M\}$, interprétée par « un chat est placé sur la case (i,j) ». Aucune autre variable n'est introduite. Dans le fichier DIMACS, $x_{i,j}$ porte le numéro $(i-1)M + j$, de 1 à NM.

1.2 Clauses

Chaque règle du jeu est traduite par une famille de clauses. « Au plus un chat » dans un ensemble de cases est exprimé en interdisant chaque paire de chats ; « au moins un chat » par une clause unique.

(1) au plus un chat par ligne : $\displaystyle\bigwedge_{i=1}^{N} \bigwedge_{1 \le j < j' \le M} (\neg x_{i,j} \vee \neg x_{i,j'})$

(2) au plus un chat par colonne : $\displaystyle\bigwedge_{j=1}^{M} \bigwedge_{1 \le i < i' \le N} (\neg x_{i,j} \vee \neg x_{i',j})$

(3) exactement un chat par région : $\displaystyle\bigwedge_{k=1}^{K} \Big(\bigvee_{(i,j) \in R_k} x_{i,j} \Big) \wedge \bigwedge_{k=1}^{K} \bigwedge_{\substack{c,c' \in R_k \\ c<c'}} (\neg x_c \vee \neg x_{c'})$

(4) pas de chats en diagonale : $\displaystyle\bigwedge_{i=1}^{N-1} \bigwedge_{j=1}^{M} \bigwedge_{\substack{j' \in \{j-1, j+1\} \\ 1 \le j' \le M}} (\neg x_{i,j} \vee \neg x_{i+1,j'})$

Deux cases voisines au sens du 8-voisinage sont soit sur la même ligne, soit sur la même colonne, soit en diagonale : les voisinages horizontal et vertical étant déjà interdits par (1) et (2), la famille (4) se limite aux diagonales. Les clauses identiques produites par plusieurs familles (par exemple deux cases d'une même région situées sur une même ligne) ne sont écrites qu'une fois.

Correction. À une valuation v on associe la configuration $S_v = \{(i,j) \mid v(x_{i,j}) = 1\}$. Chaque famille est exactement la traduction d'une règle : v satisfait (1) (resp. (2)) si et seulement si S_v contient au plus un chat par ligne (resp. colonne) ; v satisfait (3) si et seulement si chaque région contient au moins et au plus un chat de S_v ; enfin, compte tenu de (1) et (2), v satisfait (4) si et seulement si aucun couple de chats n'est adjacent. Ainsi $v \models F$ si et seulement si S_v est une solution, et $v \mapsto S_v$ est une bijection entre les modèles de F et les solutions de la grille : la formule est satisfiable si et seulement si la grille a une solution, et toute valuation témoin donne une solution.

Taille de la formule. Avant suppression des doublons, le nombre de clauses est

$$N\binom{M}{2} + M\binom{N}{2} + K + \sum_{k=1}^{K}\binom{|R_k|}{2} + 2(N-1)(M-1),$$

soit $O(NM(N+M))$ en ordre de grandeur. Le tableau 1 donne les tailles obtenues pour les grilles générées (section 1.4).

$N = M = K$	4	6	8	10	15	20
variables	16	36	64	100	225	400
clauses	74	258	638	1 253	4 341	10 547

TABLE 1 – Taille des formules CNF (nombre moyen de clauses sur 3 grilles générées par taille).

1.3 Implémentation et validation

La fonction `lire_grille` lit un fichier au format imposé et vérifie que les régions forment une partition de la grille (régions déclarées, non vides). `grille_vers_cnf` construit la liste des clauses, `ecrire_dimacs` écrit le fichier, et `convertir` enchaîne les trois. Pour la grille de la figure 1 de l'énoncé, on obtient 36 variables et 309 clauses.

La fonction `verifier_solution` contrôle directement les quatre règles du jeu sur une configuration de chats, indépendamment de la CNF. Elle a servi à valider l'encodage :

- la solution de la figure 2 de l'énoncé est licite et satisfait la CNF ;
- sur cinq grilles 4×4, pour chacune des 2^{16} valuations, « la valuation satisfait la CNF » et « la configuration associée est licite » coïncident ;

1.4 Génération de grilles

On se restreint au cas $N = M = K$. Pour garantir l'existence d'une solution, on construit *d'abord* une solution, *puis* les régions autour d'elle.

Étape 1 : placement des chats. Avec $N = M = K$, une solution place exactement un chat par ligne et par colonne : c'est une permutation σ (chat en $(i, \sigma(i))$). Les chats de deux lignes consécutives ne doivent pas être adjacents, d'où la contrainte $|\sigma(i+1) - \sigma(i)| \geq 2$. On place les chats ligne par ligne : pour chaque ligne, on essaie les colonnes libres compatibles avec le chat de la ligne précédente, dans un ordre aléatoire ; si aucune ne convient, on revient sur la ligne précédente pour y essayer la colonne suivante. Le procédé trouve un placement dès qu'il en existe un. Il n'en existe pas pour $N = 2$ (deux chats d'une grille 2×2 sont toujours adjacents) ni pour $N = 3$ (le chat de la ligne 2 devrait être en colonne 1 ou 3, et ceux des lignes 1 et 3 tous deux dans la colonne opposée) ; le générateur s'utilise donc pour $N = 1$ ou $N \geq 4$.

Étape 2 : croissance des régions. Chaque chat est le germe d'une région. Tant qu'il reste des cases libres, on choisit une région au hasard et on lui ajoute une case libre voisine (haut, bas, gauche ou droite) de l'une de ses cases. Comme la grille est connexe, toutes les cases finissent par être attribuées.

Garantie. Chaque case appartient à exactement une région, et chaque région est connexe et contient exactement un chat, son germe. Les régions forment donc une partition de la grille en $K = N$ régions non vides, et le placement de l'étape 1 respecte les quatre règles : la grille obtenue possède au moins une solution. Cette solution est écrite en commentaire dans le fichier de la grille et une graine rend chaque génération reproductible (`python generateur.py N graine sortie.txt`). Pour les 51 grilles générées ($N = 4, \ldots, 20$, trois graines par taille), `verifier_solution` confirme que la solution construite est licite.

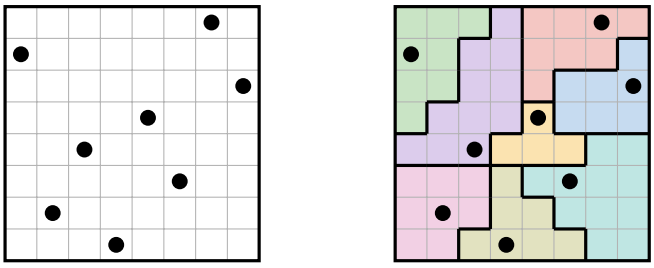


FIGURE 1 – Génération d'une grille 8×8 : placement des chats (gauche), puis croissance des régions autour de chaque chat (droite).

2 Question 2 – Solveur DPLL

Le code de cette question est dans le module `dpll.py` ; la partie 2 du notebook l'exécute.

2.1 Représentation des formules et lecture DIMACS

On reprend directement les conventions du format DIMACS :

— un *littéral* est un entier non nul, k pour x_k et $-k$ pour $\neg x_k$; l'opposé d'un littéral l est donc simplement $-l$;

— une *clause* est une liste de littéraux, une *formule* une liste de clauses ; la formule vide (aucune clause) est vraie et la clause vide $\square$ est fausse ;

— une *valuation* est un dictionnaire `variable` $\mapsto$ `bool`.

Ces structures natives de Python ne demandent aucune conversion à la lecture et rendent les opérations de l'algorithme très courtes.

La fonction `lire_dimacs` renvoie le couple (nombre de variables, clauses). Elle ignore les commentaires, exige l'en-tête `p cnf` avant la première clause, découpe les clauses sur les `0` (une clause peut donc s'étendre sur plusieurs lignes, et une ligne réduite à `0` donne la clause vide), refuse les variables dont le numéro dépasse celui de l'en-tête, et s'arrête au marqueur `%` qui termine certains fichiers de la bibliothèque SATLIB. La fonction `nettoyer` retire ensuite les littéraux répétés (`1 1 0` devient la clause x_1) et les tautologies (clauses contenant p et $\neg p$, toujours vraies). Ces deux transformations préservent l'ensemble des modèles ; la seconde est nécessaire pour que la règle du littéral pur soit correcte, et la première pour qu'une clause `1 1` soit bien reconnue comme unitaire.

2.2 Algorithmes 1 et 2

Les deux algorithmes sont écrits de façon fidèle à l'énoncé, à l'aide de quelques fonctions auxiliaires :

- `simplifier(F, l)` calcule $F[l \leftarrow 1]$: les clauses contenant l sont retirées et $\neg l$ est supprimé des autres. Comme $F[p \leftarrow b] = F[l \leftarrow 1]$ avec $l = p$ si $b = 1$ et $l = \neg p$ sinon (fonction `litteral(p, b)`), cette seule opération suffit pour les trois règles ;
- `clause_unitaire(F)` et `litteral_pur(F)` renvoient un littéral à affecter, ou `None` ;
- `random_choice(F, rng)` tire au hasard une variable présente dans F et une valeur de vérité. Le générateur aléatoire est passé en paramètre afin de rendre chaque exécution reproductible (option `--graine`).

Algorithme 1 (`dpll_statut`). Il renvoie seulement `True` (SAT) ou `False` (UNSAT) et suit les lignes 1 à 13 : clause vide, formule vide, clause unitaire, littéral pur, puis décision aléatoire avec retour arrière sur la valeur opposée.

Algorithme 2 (`dpll`). Il renvoie `(True, v)` ou `(False, None)`. La valuation est construite *en remontant* les appels récursifs : chaque niveau ajoute à v le littéral qu'il a affecté ($v \cup \{l\}$ ou $v \cup \{p \leftarrow b\}$). Les lignes 5–10 et 11–16 de l'énoncé ayant le même corps, elles sont regroupées : on cherche d'abord une clause unitaire, à défaut un littéral pur, puis on fait l'appel récursif. La décision (lignes 17–26) est une boucle sur les deux valeurs b puis $\bar{b}$.

Modèle complet. La valuation renvoyée ne contient que les variables effectivement affectées. Une variable absente a disparu de la formule sans être choisie, parce que toutes ses clauses étaient déjà satisfaites : n'importe quelle valeur convient. La fonction `completer` lui donne la valeur faux et produit le modèle au format DIMACS $[\pm 1, \ldots, \pm n]$. La fonction `verifier_modele` contrôle enfin que chaque clause contient au moins un littéral vrai.

Correction. Chaque règle préserve la satisfiabilité : une clause unitaire $\{l\}$ impose l ; si l est pur, tout modèle de F reste un modèle après passage de l à vrai, donc F et $F[l \leftarrow 1]$ sont équisatisfiables ; la décision explore les deux valeurs. Chaque appel récursif affecte une variable, qui disparaît de la formule : la récursion termine, avec une profondeur au plus $n+1$ (n variables), d'où la limite de récursion relevée à 100 000.

Ligne de commande. `python dpll.py fichier.cnf [--algo 1|2] [--graine G]` affiche `s SATISFIABLE` ou `s UNSATISFIABLE` et, pour l'algorithme 2, la ligne `v ... 0` du modèle, vérifié avant affichage. Le temps de calcul est écrit sur la sortie d'erreur, conformément au format qui sera imposé à la question 6.

2.3 Validation

Comparaison avec la force brute. Sur 500 formules aléatoires (1 à 8 variables, 0 à 30 clauses de 1 à 3 littéraux, avec littéraux répétés et tautologies possibles), le statut renvoyé par les deux algorithmes coïncide toujours avec celui obtenu par énumération des 2^n valuations, et chaque modèle renvoyé par l'algorithme 2 est vérifié (154 formules SAT, 346 UNSAT).

Instances des enseignants. Les 41 fichiers du dossier `public-instances` ont été lancés avec les deux algorithmes, chacun dans un processus séparé avec la limite de 60 s de la compétition ; le statut est comparé à `expected.json` et les modèles sont vérifiés. Les 11 petites instances (tests d'interface `smoke`, `unit`, `cli` et cas limites `edge` : formule vide, formule sans clause, clause vide, tautologie, littéraux répétés, littéraux tous négatifs) sont toutes correctement résolues. Le tableau 2 donne les résultats sur les 30 instances de référence.

instance	var.	clauses	attendu	algo 1	algo 2	instance	var.	clauses	attendu	algo 1	algo 2
01	48	261	SAT	0,04	0,04	16	42	133	UNSAT	0,15	0,14
02	90	300	SAT	0,05	0,06	17	100	850	SAT	0,96	0,97
03	63	168	UNSAT	–	–	18	459	7 054	SAT	7,60	7,53
04	180	800	SAT	0,10	0,09	19	729	22 060	SAT	4,80	4,80
05	50	218	UNSAT	0,52	0,53	20	50	218	SAT	0,23	0,22
06	1 363	3 034	SAT	–	–	21	510	3 065	SAT	0,42	0,42
07	225	1 280	SAT	0,13	0,13	22	100	850	UNSAT	4,86	4,96
08	100	900	UNSAT	1,67	1,70	23	350	1 149	SAT	0,17	0,16
09	50	218	SAT	0,08	0,07	24	50	218	UNSAT	0,40	0,39
10	50	218	UNSAT	0,45	0,46	25	50	218	UNSAT	0,47	0,46
11	729	21 844	SAT	4,39	4,38	26	100	800	SAT	0,09	0,08
12	522	11 636	SAT	–	–	27	181	3 151	SAT	–	–
13	61	581	SAT	0,59	0,58	28	50	218	UNSAT	0,26	0,26
14	68	270	SAT	0,06	0,07	29	150	545	SAT	0,23	0,22
15	100	850	UNSAT	2,70	2,70	30	100	850	UNSAT	6,82	6,94

TABLE 2 – Instances `public-01` à `public-30` : temps d'exécution en secondes (processus complet, démarrage de Python compris, graine 0), « – » = dépassement des 60 s. Toutes les réponses obtenues sont correctes et tous les modèles sont vérifiés.

Grilles Meowdoku. Sur les grilles générées à la question 1 ($N = 4, 6, 8, 10, 12$), l'algorithme 2 trouve une valuation en moins de 0,1 s ; la configuration de chats associée est validée par `verifier_solution`.

2.4 Performances

Bilan. Les deux algorithmes ne donnent jamais de réponse fausse : 37 instances sur 41 sont résolues dans la limite de temps, les 4 autres (`public-03`, `06`, `12`, `27`) dépassent les 60 s. Sur les petites instances, le temps mesuré ($\approx$ 0,05 s) est essentiellement celui du démarrage de l'interpréteur.

Algorithme 1 contre algorithme 2. Les temps sont presque identiques : avec la même graine, les deux algorithmes font exactement les mêmes choix et explorent le même arbre ; la construction de la valuation ne coûte qu'une affectation de dictionnaire par appel récursif réussi, négligeable devant la simplification de la formule. Les petits écarts observés (par exemple 4,86 et 4,96 s sur `public-22`) viennent du bruit de mesure, les instances étant lancées quatre par quatre en parallèle. Pour la même raison, les temps absolus varient d'une exécution à l'autre selon la charge de la machine ; seuls les ordres de grandeur sont significatifs.

Coût d'un appel. Soit $|F|$ la taille de la formule (somme des longueurs des clauses). Chaque appel récursif coûte $O(|F|)$ pour `simplifier`, qui recopie la formule, $O(|F|)$ pour chercher une clause unitaire, et $O(|F| + n \log n)$ pour chercher un littéral pur (ensemble des littéraux puis tri). Comme chaque propagation unitaire ou élimination de littéral pur est un appel récursif complet, propager une chaîne de m clauses unitaires coûte $O(m\,|F|)$. La mémoire est en $O(n\,|F|)$ dans le pire cas, puisque chaque niveau de la pile garde sa copie de la formule.

Nombre d'appels. Dans le pire cas, l'arbre de recherche a 2^n feuilles. Le choix aléatoire de la variable de décision ne tient aucun compte de la structure de la formule : sur les instances UNSAT, qui obligent à parcourir tout l'arbre, et sur les grandes instances structurées (`public-06` : 1 363 variables ; `public-12` : 11 636 clauses), le nombre de décisions explose. Les grilles Meowdoku sont au contraire faciles : dès qu'un chat est placé, les clauses binaires « au plus un » propagent immédiatement l'interdiction de toute sa ligne, sa colonne, sa région et son voisinage.

Ces mesures fixent les deux axes d'amélioration traités dans la suite : réduire le nombre de décisions par une heuristique de branchement (questions 4 et 5), et réduire le coût d'un appel en évitant de recopier la formule à chaque étape (question 6).

3 Question 3 – Résolution d'une grille Meowdoku

Le programme `resoudre.py` prend un fichier grille et produit un fichier solution :

```
python resoudre.py grille.txt solution.txt [--cnf fichier.cnf] [--graine G]
```

3.1 Étapes

La fonction `resoudre` enchaîne les programmes des deux questions précédentes.

1. **Lecture et conversion** (question 1) : la grille est lue et contrôlée par `lire_grille`, puis convertie en fichier DIMACS par `grille_vers_cnf` et `ecrire_dimacs` (la grille n'est lue qu'une fois). Ce fichier est conservé (par défaut à côté du fichier solution, avec l'extension `.cnf`).
2. **Résolution** (question 2) : le fichier DIMACS est relu par `lire_dimacs` et la formule est passée à l'algorithme 2 (`dpll`), qui fournit une valuation témoin. La recherche de solution est ainsi entièrement déléguée au solveur, à travers le format DIMACS, comme demandé. La graine (0 par défaut) rend le résultat reproductible.
3. **Décodage** : les variables vraies de la valuation sont traduites en cases par `chats_depuis_valuation`, en inversant la numérotation $x_{i,j} \mapsto (i-1)M + j$ de la question 1. Par la bijection établie à la question 1, ces cases forment une solution de la grille.
4. **Écriture** (`ecrire_solution`) au format de la figure 2 de l'énoncé : copie à l'identique du fichier d'entrée, puis une ligne « région ligne colonne » par région, dans l'ordre de la ligne `ZONES`, puis la grille où les chats sont marqués par `*` et les autres cases par `.`.
5. **Vérification** : le fichier solution écrit est relu (`lire_solution`) et l'on contrôle que le dictionnaire donne un chat par région déclarée, que chaque chat est bien dans la région annoncée, que le dictionnaire et la grille de `*` désignent les mêmes cases, et enfin que cette configuration respecte les quatre règles du jeu (`verifier_solution` de la question 1, indépendante de la CNF). En cas d'échec, le programme s'arrête avec un message d'erreur.

La vérification porte sur le fichier produit et non sur la valuation : elle contrôle ainsi l'ensemble de la chaîne, y compris le décodage et l'écriture. Une grille sans solution, exclue par l'énoncé, provoquerait une réponse UNSAT du solveur et un message d'erreur.

3.2 Résultats

Pour la grille de la figure 1 de l'énoncé, le fichier produit est identique à celui de la figure 2 :

```
A 3 2                    . . * . . .
B 2 5                    . . . . * .
C 1 3                    . * . . . .
D 5 6                    . . . * . .
E 4 4                    . . . . . *
F 6 1                    * . . . . .
```

Le programme a ensuite été lancé sur les 51 grilles générées à la question 1, avec la limite de 60 s par grille (tableau 3). Toutes les solutions écrites passent la vérification. Pour certaines grilles, la solution trouvée diffère de celle construite par le générateur : ces grilles ont plusieurs solutions.

N	graine 0	graine 1	graine 2	N	graine 0	graine 1	graine 2
4–12		$< 0{,}2$ (toutes)		17	0,57	1,49	0,51
13	0,16	4,75	0,16	18	12,51	0,67	16,12
14	0,23	0,25	0,24	19	23,05	1,24	–
15	0,56	0,29	0,42	20	0,99	1,01	57,93
16	48,62	2,99	0,67				

TABLE 3 – Résolution des grilles générées : temps en secondes (processus complet, démarrage de Python compris), « – » = dépassement des 60 s.

Jusqu'à $N = 15$, toutes les grilles sont résolues en moins de 5 s, le plus souvent en moins d'une seconde. Au-delà, le temps devient très variable d'une grille à l'autre : une grille sur 51 (`meow_19_2`) dépasse les 60 s et deux autres s'en approchent (`meow_16_0` : 48,6 s ; `meow_20_2` : 57,9 s), si bien que le nombre de grilles résolues peut varier d'une exécution à l'autre selon la charge de la machine : le solveur utilisé est le DPLL de base de la question 2, qui choisit au hasard la variable de branchement et sa valeur. L'amélioration de ces temps fait l'objet des questions 4 à 6.

4 Question 4 – Heuristiques de branchement

Le code de cette question est dans le module `heuristiques.py` ; la partie 4 du notebook l'exécute.

4.1 Algorithme 3

La fonction `dpll_h(F, choix)` reprend exactement l'algorithme 2 (`dpll`) et les mêmes fonctions auxiliaires (`simplifier`, `clause_unitaire`, `litteral_pur`) ; seule la ligne 17 change : la décision (p, b) est donnée par `choix(F)` au lieu de `random_choice`. Une heuristique est donc simplement une fonction qui reçoit la formule courante et renvoie une variable et une valeur de vérité.

4.2 Choix des heuristiques

Nous avons retenu quatre heuristiques. Elles respectent les contraintes de l'énoncé : H_3 sélectionne un littéral, H_4 tient compte de la taille des clauses, et H_2 à H_4 sont dynamiques, c'est-à-dire recalculées sur la formule courante à chaque décision (tableau 4)

		variable p	valeur b	fonction
H_1	PVL	plus petite variable de F	1	`h1_pvl`
H_2	VPPM	$\arg\max_p \#(p, F)$	1 si $\#^+ \geq \#^-$	`h2_vppm`
H_3	LP	$\lvert l\rvert$ avec $l = \arg\max_l occ(l, F)$	1 si $l = p$, 0 si $l = \neg p$	`h3_lp`
H_4	VFCM	$\arg\max_p \#_k(p, F)$, $k = \min_{C \in F} \lvert C\rvert$	1 si $\#_k^+ \geq \#_k^-$	`h4_vfcm`

TABLE 4 – Heuristiques implémentées (versions dynamiques) : première variable libre (PVL), variable la plus fréquente avec polarité majoritaire (VPPM), littéral le plus fréquent (LP), variable la plus fréquente dans les clauses de taille minimale (VFCM).

— H_1 ne fait aucun calcul de fréquence : c'est la référence déterministe, qui mesure ce qu'apporte une vraie heuristique par rapport à un ordre arbitraire. Sur les grilles Meowdoku, l'ordre des variables est celui des cases, ligne par ligne, et la valeur 1 revient à poser un chat sur la première case libre.

— H_2 : une variable fréquente simplifie beaucoup de clauses d'un coup, et la polarité majoritaire satisfait le plus grand nombre de clauses, ce qui augmente les chances de trouver un modèle sans retour arrière sur les instances SAT.
— H_3 applique la même idée au niveau du littéral : elle choisit le littéral qui satisfait le plus de clauses. Elle diffère de H_2 lorsqu'une variable est fréquente mais avec des polarités équilibrées.
— H_4 (MOMS) privilégie les clauses les plus courtes : ce sont les plus proches de devenir unitaires ; brancher sur leurs variables déclenche la propagation unitaire, ou fait apparaître une clause vide, plus tôt dans l'arbre.

Pourquoi des versions dynamiques ? Dans notre implémentation, chaque appel récursif recopie déjà la formule en $O(|F|)$ (`simplifier`). Recompter les occurrences à chaque décision coûte aussi $O(|F|)$ (plus un tri des variables en $O(n \log n)$) : la version dynamique ne change donc pas l'ordre de grandeur du coût d'un appel, alors qu'elle reflète la formule réellement restante. Une version statique n'économiserait qu'un facteur constant.

Égalités. En cas d'ex aequo, on retient la plus petite variable (puis la polarité positive pour H_3). Le solveur est ainsi entièrement déterministe : il n'a plus besoin de graine. Sur la formule d'exemple de l'énoncé, $(x_1 \vee x_2 \vee \neg x_4) \wedge (x_2 \vee x_3) \wedge (x_1 \vee \neg x_2 \vee x_4 \vee \neg x_5)$, on obtient $x_1 \leftarrow 1$ pour H_1 et H_3 ($occ(x_1) = occ(x_2) = 2$), et $x_2 \leftarrow 1$ pour H_2 ($\#(x_2) = 3$) et H_4 ($k = 2$, clause $x_2 \vee x_3$).

4.3 Validation

L'énoncé demande que les heuristiques donnent des résultats conformes à l'algorithme 2. Nous reprenons les trois tests de la question 2.

Comparaison avec la force brute. Sur 500 nouvelles formules aléatoires (mêmes paramètres qu'à la question 2), les quatre heuristiques renvoient toujours le statut obtenu par énumération des 2^n valuations, et tous les modèles sont vérifiés (147 SAT, 353 UNSAT).

Instances des enseignants. Les 41 instances ont été lancées avec chaque heuristique, dans les mêmes conditions qu'à la question 2 (processus séparé, limite de 60 s, statut comparé à `expected.json`, modèles vérifiés). **Aucune réponse n'est fausse.** Les 11 petites instances et les instances faciles sont résolues en moins d'une seconde par toutes les heuristiques ; le tableau 5 donne les instances pour lesquelles au moins un solveur dépasse une seconde.

Grilles Meowdoku. Les 51 grilles de la question 1 ont été résolues par chaque heuristique à partir de leur fichier DIMACS, avec la même limite ; la valuation est décodée en chats puis contrôlée par `verifier_solution`. **Les 51 grilles sont résolues par les quatre heuristiques et toutes les solutions sont licites**, contre 50 sur 51 avec l'algorithme 2 (tableau 3), dont deux à moins de 12 s de la limite. Le temps reste inférieur à 1,3 s pour toutes les grilles, sauf deux : `meow_17_1` pour H_2, H_3 et H_4 (25, 25 et 11 s) et `meow_20_1` pour H_1 (28 s).

Premiers constats. Ces mesures servent ici à la validation ; leur analyse détaillée (nombre de décisions, de retours arrière, coût des heuristiques) relève de la question 5. On note déjà que H_4 résout le plus d'instances (40 sur 41, seule `public-03` résiste) et qu'elle est la plus rapide sur les instances UNSAT (`public-15`, `22`, `30`), où il faut parcourir tout l'arbre : brancher sur les petites clauses provoque plus tôt les conflits. À l'inverse, H_2 (`public-04`, `11`, `21`) et H_3 (`public-11`) échouent sur des instances SAT que le tirage aléatoire résout : une heuristique déterministe qui se trompe de branche se trompe à chaque fois, alors que le hasard peut l'éviter.

instance	attendu	algo 2 (Q2)	H_1	H_2	H_3	H_4
03	UNSAT	–	–	–	–	–
04	SAT	0,09	0,23	–	12,01	0,54
06	SAT	–	1,51	5,77	–	1,07
08	UNSAT	1,70	0,08	0,31	0,37	0,09
11	SAT	4,38	4,69	–	–	6,70
12	SAT	–	–	10,32	2,39	41,31
15	UNSAT	2,70	5,00	0,34	2,12	0,10
18	SAT	7,53	59,15	12,97	21,15	0,78
19	SAT	4,80	6,63	4,23	17,59	3,82
21	SAT	0,42	0,50	–	0,41	0,42
22	UNSAT	4,96	1,77	0,49	0,58	0,19
27	SAT	–	18,11	39,53	27,13	29,60
30	UNSAT	6,94	3,50	0,92	1,37	0,16
résolues / 41		37	39	37	38	40

TABLE 5 – Instances `public-XX` où au moins un solveur dépasse 1 s : temps en secondes (processus complet), « – » = dépassement des 60 s. La colonne « algo 2 » reprend le tableau 2 (même exécution du notebook).

Usage des assistants IA

Pour chaque question, nous avons d'abord défini notre stratégie de résolution. Nous avons ensuite utilisé Claude (Opus 5.5, effort moyen) pour générer une première version du code et du rapport, que nous avons relus, testés et repris.